MeetLoop 开发与学习主计划

v3.0 | 2026-10-29 审核版 | Swift / iOS + React Native 主项

可以开工。方向保留；本版统一阶段与工时，提前 UIKit 和独立编码，补足跨端边界与求职验收。

产品保持活动列表 → 详情 → 报名 / 取消 → 我的报名。Swift / iOS 为第一主项，TypeScript / RN 为第二主项，Kotlin / Android 为宿主辅项，Go 为最小可靠服务。项目与 AgentDesk 的代码、数据和部署独立。

当前已准备	尚未完成
独立公开仓库、版本管理、计划源文件、课程入口、阶段任务与验收标准。	应用工程、业务实现、真机 / Release 验证、自动化测试、CI 与真实用户反馈。
本版 Markdown、HTML、PDF 内容来自同一份计划源，附官方 JD 能力审核。	你的 Mac / Xcode 环境与当前独立编码水平，需要 M0 亲自验收。

这不是所有前端或所有全栈岗位的通用套餐。它直接训练移动端与跨端；通用 SDE 还需要公共基础，Web 前端需要浏览器专项。AgentDesk 的 Web 控制台可以训练可迁移的技能，两个项目不需要业务互通。

唯一执行口径

docs/plans/plan.source.json 为内容源，plan.md、plan.html 与本 PDF 为同版快照。START_HERE 负责起步，PROGRESS 记录证据。旧 v2.2 / v2.3 PDF 只留作历史参考；阶段和工时一律以本版为准。

阶段总表与面试出口

工时从现在起累计，不是额外叠加；已掌握部分必须通过实操才可跳过

阶段	本阶段	累计	完成后能做什么
M0 基线与环境	10-15h	10-15h	能独立写筛选函数；工程可构建，记录工具链。
M1 本地 iOS 小闭环	45-65h	55-80h	列表、详情、报名、我的报名；状态单一且可测试。
M2 iOS 数据与可靠性	65-90h	120-170h	网络、缓存、取消、并发与内存故障能复现和修复。
M3 UIKit 与最小 Go	60-90h	180-260h	原生专项、服务端正确性与第一份面试作品。
M4 JS / TS / React	55-80h	235-340h	独立写表单与异步状态，区分 DOM 与 RN。
M5 RN 与 iOS 集成	80-110h	315-450h	报名模块嵌入 iOS，契约、原生能力、Release 可验证。
M6 Kotlin / Android 宿主	55-80h	370-530h	生命周期与重建正确，同一个 RN 模块在 Android 运行。
M7 性能、发布与求职	80-120h	450-650h	可复现实验、发布演练、用户反馈与完整技术讲解。

M0-M3 合计 180-260h，是开始面试部分 iOS 日常实习的准备区间，不是录用线。M0-M7 合计 450-650h，训练完整移动端 / 跨端工程链路。估时包括补课、实现、测试与正常返工；未做能力基线测试，区间仍有不确定性。

顺序：M0 → M1 → M2 → M3 → M4 → M5 → M6 → M7。测试、日志、可访问性与文档从首次相关实现就开始；M7 是加深，不是首次补质量。

M0-M1：把 Swift 与状态写明白

M0 10-15h；M1 45-65h | 先做本地闭环，不接真实账号和后端

先修与学习

用 100 Days of SwiftUI 的语言基础与最初项目作为主课；Swift 官方指南查 Optional、结构体、枚举、协议、闭包、集合。每天课程概念立即迁移到活动业务；能过关的课程章节可以跳过。

按这个顺序开发

- M0：安装 / 确认正式工具链，记录 macOS、Xcode 与 Swift；写 Activity 模型和筛选函数，首次 Git 提交。
- M1：三条内存活动 → 列表和详情 → 单一数据源报名 / 取消 → 我的报名 → 空状态与错误提示。先把身份、状态与规则分离。
- 给容量、重复报名、取消后恢复写业务断言；UI 不能靠多份独立数组碰巧同步。支持基本动态字体与按钮可访问标签。

过关后你应能

关闭教程，在 45 分钟内增加“只显示可报名活动”并保证列表与详情一致；解释值与引用、Optional、绑定、状态所有者和视图身份。展示重复点击不会重复增加本地报名记录。

> **首关不要求缓存、MVVM 框架、RN 或后端。能用简单结构解释状态，比提前造架构更重要。Mac 未就绪时只能先做纯 Swift / Git 练习，界面验收仍需 Xcode。**

100 Days of SwiftUI / Swift 语言指南

M2：网络、并发、缓存与排错

65-90h | 进入条件：本地四步链路独立完成

开发顺序	需要掌握	验收实验
Repository 与可替换数据源	协议边界、错误分类、可测试依赖。	用 Mock 注入空数据、错误和延迟；UI 的加载 / 空 / 成功 / 失败状态清楚。
URLSession 与并发	async / await、MainActor、任务取消与生命周期、HTTP 语义。	旧请求晚到不能覆盖新页面；切页取消；区分超时与服务端拒绝。
持久化与会话	缓存来源、时效与迁移；令牌不写入日志，秘密进 Keychain。	离线重启可读已有内容；登录失效有恢复路径；不要离线假装报名已成功。
内存和性能	ARC、闭包捕获、引用链；Memory Graph、Instruments。	复现并修复一个循环引用或卡顿，保留触发步骤与前后数据。

网络阶段先用可控本地假服务或测试替身，不依赖生产 API。数据层稳定后，再由 M3 的 Go 接口替换。并发安全以当前工具链检查结果和可复现实验验证，不靠强行关闭检查通过编译。

过关能力

画出请求、取消和 UI 更新的时序；从日志 / 断点定位一次旧响应覆盖；解释 weak、actor 隔离和持久化取舍。保存至少一个业务测试和一个故障回归。

Swift 语言指南 / Stanford CS193p

M3：UIKit、最小 Go 与首轮面试

60-90h | 不等 RN / Android 完成才申请 iOS 实习

工作包	开发内容与验收
UIKit 约 25-40h	用 Apple Today 学列表、导航、Auto Layout、控制器生命周期与复用；把 MeetLoop 一个列表或详情做成 UIKit，复用数据层。修复复用错图 / 退出后回调。
Objective-C 阅读实验	在 UIKit 工时内留约 6-10h：读一个小型 ObjC 类，从 Swift 调用；解释 ARC 和 bridging。深入 runtime 仅按目标 JD 加练。
Go 约 35-50h	单服务 + PostgreSQL；活动、报名和我的报名接口；身份认证、资源授权、输入校验、迁移、唯一约束和事务。选择成熟登录方案，不自造密码算法。
报名正确性	并发报名不能超容量；重复请求用幂等键；取消不会错误归还名额；写入后响应丢失时客户端查询事实，不能盲目重试。

Go 弱化的是服务规模，不是数据正确性。写 API 契约、结构化日志、数据库备份 / 恢复说明和核心集成测试；不引入微服务、消息队列、Kubernetes。

M3 面试出口

- 一个可复现运行的 iOS 产品；UIKit 专项；一例内存 / 并发排错；一次跨请求到 SQL 的正确性验证；3 分钟演示。
- 60 分钟独立增加报名规则，说明客户端提示与服务端约束各自职责；如后端进度落后，简历只写实际完成范围。

Apple Today / UIKit / A Tour of Go

PostgreSQL Tutorial

M4-M5：React 基础到 RN 主项

M4 55-80h；M5 80-110h | 先独立 RN 页面，再嵌入原生宿主

顺序	开发任务	过关能力
M4 JS / TS	闭包、Promise / 事件循环、模块；联合类型与运行时校验。	不用 any 掩盖报名状态；解释类型检查不能验证服务器输入。
M4 React	组件、状态、单向流、表单、effects 与 cleanup；先做小练习。	独立改规则并修复 stale closure；区分 DOM / CSS 与 RN 组件 / 样式。
M5 独立 RN	报名表单、状态、键盘、导航、网络错误；锁定当前稳定版本。	真机 / 模拟器跑通，离开页面后任务与订阅正确清理。
M5 嵌入 iOS	输入 activity_id 和受限会话能力；输出 completed / cancelled / needs_login / pending。	回调至多一次；原生负责导航；JS 重载、重复打开与超时结果可解释。
M5 原生能力	实现系统分享一条调用链，学习 Codegen / TurboModule / Fabric / JSI 各自职责。	追踪 JS → 原生 → API → 回调；构建 Release，在无 Metro 时可启动。

教程按项目实际 RN 版本使用，新架构与旧 Bridge 资料分开。先采用官方现有应用集成路径；Expo 可以比较，但不要同时维护多套构建路线。RN 到 Lynx 的 React / TS 知识可迁移，宿主、样式、能力接口与工具链仍需另学，当前不加 Lynx 分支。

TypeScript Handbook / React Learn

React Native 现有应用集成 / React Native 原生模块

M6：Kotlin 与 Android 辅项

55-80h | 进入条件：RN 模块和 iOS 宿主契约稳定

做一个小而完整的宿主

- 学习 Kotlin 空安全、数据类、sealed 类型、协程 / Flow；Compose 状态、ViewModel 与 Activity 生命周期。
- 原生实现活动列表 / 详情，复用接口契约；接入同一个 RN 报名模块。无需复制全部 iOS 页面。
- 处理旋转、进程重建、返回键、重复打开、退出后取消和权限拒绝；状态恢复不得重复触发报名。
- 完成 Android Release 安装与运行，记录 JDK / Gradle / AGP 兼容组合；从 JS 错误和原生日志定位跨层问题。

过关后你应能

从冷启动到报名回调画出生命周期；解释 Flow 收集何时开始和取消；制造一次宿主重建并验证 UI、网络和业务结果一致。用同一宿主契约解释两端差异。

该阶段训练跨端宿主与 Kotlin 工程基础，不等同于资深 Android 深度。若投原生 Android 主岗，应另补 Java、系统组件、平台性能与该 JD 的专项，不能只凭 RN 模块就声称覆盖。

Android Basics with Compose / React Native 现有应用集成

M7：工程深度与作品验收

80-120h | 从第一天积累证据，到此集中验证与补齐

交付	验收标准
测试与 CI	关键业务单元、API 集成、核心端到端流程分层；锁依赖；CI 从干净检出可构建和执行对应检查。区分模拟器与真机覆盖。
性能与内存	选一个真实瓶颈；固定设备、系统、构建、数据和网络；记录冷暖启动、列表流畅度 / 内存中的相关指标及重复测量。
发布与恢复	iOS Archive 和 Android Release；签名、配置、权限、日志脱敏；一次坏版本 / API 兼容失败的恢复演练。外部分发按账号条件另验。
产品质量	深链、可访问性、键盘与弱网；两轮 2-3 人体验，记录任务成功与问题，不伪造留存或商业指标。
面试作品包	运行说明、架构图、API / 宿主契约、3 个故障案例、3 分钟演示、一次现场需求变更与独立解释。

App Store 上架、支付、IM、地图、推送和社交推荐均不属于首版必做。面试主线优先证明状态、数据和生命周期正确，功能数量不作为“扎实”的指标。

可以写进简历的边界

只写能解释和重现的行为与测量。不要把“集成了 RN”写成自研跨端引擎；不要把一台机器的并发实验写成大规模生产经验；不要把 AI 生成但尚未理解的模块当作独立成果。

岗位覆盖与 Web / 全栈补强

本版按官方招聘样本调整；详细来源与证据强弱见 JOB_REQUIREMENTS_AUDIT.md

岗位目标	覆盖判断	还要补 / 证明什么
iOS 实习 / 初级	M0-M3 直接相关；M7 增加工程深度。	Swift + UIKit、基础 ObjC 阅读、独立排错、数据结构与项目贡献；按 JD 核对在读资格。
RN / 移动跨端	M4-M6 核心对应，建立在原生基础上。	React / TS、原生集成、平台差异、性能和 Release；并非所有团队都用 RN。
Web 前端	MeetLoop 主线覆盖不足。	DOM、CSS、浏览器、Web 网络安全、组件测试与 Web 性能。AgentDesk A4 提供入门实践。
Full-stack / 通用 SDE	两项目 + 公共基础提供作品和练习。	一条完整 Web 到 DB 的生产式链路、设计取舍、测试部署、算法面试；不声称覆盖所有后端专项。

若以后以 Web / 全栈为主要投递方向

在 A4 / A7 后另开约 80-140h 专项：响应式与语义布局、浏览器渲染 / 缓存、组件与端到端测试、Web 性能测量、登录与 CSRF / XSS；再按真实 JD 选 SSR / Next.js 或 Node 服务中的一个。主线不预装全家桶，额外工时不藏在 450-650h 里。

MDN Web 核心课程 / Full Stack Open（选学）

双线时间安排与求职节奏

总预算每周 21 小时起；课程、调试、测试、文档和面试都算在内

时期 / 条件	MeetLoop	AgentDesk	公共基础
现在至首轮 iOS 面试准备	13h	5h	3h
M3 过关后的默认安排	8h	10h	3h
实习 / 考试导致时间减少	按真实余量保留一个主项	维持小步或暂停新功能	保留每周复盘

从 2026 年 11 月到 2027 年 2 月底约 17 周：约 221h 给 MeetLoop、85h 给 AgentDesk、51h 给公共基础。对应目标是 M3 的中低区间和 A1 完成；不是四个月内做完双项目。若起点不足或春节暂停，按关卡顺延。

两条主线各为 450-650h，合计 900-1300h；每周项目时间 18h，粗算约 50-73 个有效周。按 M3 过关即从 13/5 切到 8/10、完成一项后暂不重分配工时估算，约 52-75 周；再加 10%-20% 中断余量，约 14-21 个日历月。18 个月可争取中档完成，高档或大幅中断会更久。

投递不等全部完工

- 2026 年 12 月整理实习复盘与简历，开始关注官方岗位；M2 后可试投并记录面试缺口，M3 作为首轮目标。
- 春季 2027 上海优先、4-5 天 / 周、连续至少 3 个月；毕业时间、在读身份和具体到岗日期逐项核对，不因作品完成就默认有资格。
- 美国岗位按届别及在读要求提前查看；具体实习或工作资格以学校与雇主确认为准。本次不推断个人资格。

月度只调整工时与下一关。不要因岗位热词每周更换技术栈；岗位复核在 M3、A5 和正式投递前进行。

两个项目之外必须持续训练

公共 3h / 周单独记账；同一知识不在两个项目重复计时

能力	训练安排	能证明什么
编码与数据结构	每周 90 分钟：数组 / 字符串 / 哈希、栈队列、二分、树与图、堆；再按 JD 补基础 DP。	能说复杂度、列边界、独立写代码；隔周重做并换条件。
网络 / OS / SQL	每周 60 分钟轮换一个主题：HTTP / TLS / DNS、进程线程与并发、事务索引与查询计划。	能跟踪客户端到数据库的一次请求；用实验解释取消、锁或超时。
表达与复盘	每周 30 分钟：3 分钟演示、一次故障讲解或英文方案说明。	讲清问题、选择、证据与限制；区分自己完成与 AI 辅助。

Git 分支、差异审查、冲突与回滚；终端、Linux 进程 / 端口 / 日志；单元与集成测试、CI、依赖锁定和密钥管理，随着实际项目用到时练。不要把“会框架”当成已经会 SDE 基础。

每四周一次独立复测

- 先不开 AI，用 45-90 分钟完成一个新规则或工具；允许查官方文档。随后可以让 AI 审查差异。
- 拿一个未提前看答案的故障，写出复现步骤、假设、修复与回归。未通过就缩小任务补练，不能只重新播放课程。
- 普通业务与数据结构面试仍要练；数学不强不妨碍这条应用工程路线，但不能跳过复杂度、基础概率和测量思维。

Pro Git / MIT 6.006 算法课程

MIT Missing Semester / PostgreSQL Tutorial

课程怎样配合开发

课程是一项任务的工具，不是先看完三套课再开始写项目

阶段	主资料	看完后立即做什么
M0-M1	100 Days of SwiftUI + Swift 语言指南。	基础检查点迁移到活动模型和状态；100 天是课程设计，不是能力或日历保证。
M2	CS193p 2025 选相关讲次与作业；Swift 并发官方资料。	只解决模型、状态、身份、异步等当前疑问，不与主课全量并行。
M3	Apple Today + UIKit 文档；Go Tour + PostgreSQL 教程。	移植一个真实页面；写报名事务并发测试。Today 入口需浏览器 JS，本次未完整验课。
M4-M5	MDN JS、TS Handbook、React Learn、RN 官方集成与原生模块。	先表单和异步，再 RN 页面，再宿主集成；不抄旧构建配置。
M6-M7	Android Basics with Compose 与实际问题对应官方资料。	重建、取消、性能和发布实验；记录实际工具链与来源版本。

100 Days 是从基础逐步进入完整应用的入门主课；完整吸收会建立可用基础，但不会自动覆盖 UIKit、RN、Android、工程发布与所有面试知识。学到能迁移就回项目，之后按缺口补后续章节。

100 Days of SwiftUI / Swift 语言指南

Stanford CS193p / Apple Today / UIKit

TypeScript Handbook / React Learn

React Native 现有应用集成 / Android Basics with Compose

起步、复盘与防止走偏

今日先做 M0；具体时间块见 START_HERE.md

- 先用 45 分钟写一个不依赖 AI 整段生成的 Activity 筛选 / 容量规则练习。记录不会的语言点；不要因为有四个月实习就跳过验证。
- Mac 可用：建立最小 SwiftUI 工程并显示三条活动；Mac 未就绪：完成纯 Swift 和 Git，等待环境后再计 UI 关卡。
- 第一周按 13h：诊断 / 环境 3h、课程与练习 3h、本地模型和列表 5h、测试 / 复盘 2h；若 M0 未过，后续工时补 M0。
- 每次提交一项可解释的变化；每阶段填写 PROGRESS 中的演示、测试、故障、独立变更证据。不要提前打完成勾。

开始新技术前的三个条件

当前阶段核心任务能独立完成；关键故障有回归；下一阶段确实需要新技术。满足后再加 RN、Android 或框架。课程进度、生成代码量和收藏链接数不替代这三个条件。

开工准备已足够。应用实现、环境验收、真实评测与独立掌握仍未完成；本次审核只是让学习顺序和通过标准更可靠，并不保证录用。